Pizza Shapes Mini App

Pizza Shapes is a multiplayer, turn-based strategy game built as a Farcaster Mini App on Base mainnet. Inspired by Dots and Boxes, players create pizza slice shapes on a square grid to compete for onchain $PIZZA rewards.

Token Address: `0xa821f2ee19f4f62e404c934d43eb6e5763fbdb07`

---

1. High-Level Overview

- **Game Type:** Turn-based, multiplayer (2–6 players)
- **Network:** Base Mainnet
- **Platform:** Farcaster Mini App
- **Core Objective:** Capture the most pizza slice shapes
- **Mechanics:** Players take turns drawing lines between dots. Completing the THIRD side of a pizza slice shape captures it and grants an extra turn. Pizza slices can be small or large, and all valid shapes count once when completed.

---

2. Game Inspiration

Pizza Shapes is based on Dots and Boxes with a twist: - Players form pizza slice shapes (triangles) instead of squares - Diagonal lines are allowed - Any size pizza slice is valid - Reference video: https://www.reddit.com/r/oddlysatisfying/comments/1qqiy00/

---

3. Board & Grid Setup

Square Grid: - Example sizes: 4x4 (beginner), larger grids increase strategy - **Player Interaction:** Click/tap dots to connect; valid move = adjacent dot connection - **Allowed Lines:** Horizontal, vertical, diagonal

---

4. Pizza Slice Shapes

- **Definition:** Any closed 3-sided shape (triangle)
- **Capture Rule:** Third side completion captures the slice
- **Extra Turn:** Capturing a slice grants an immediate extra turn
- Supports small & large slices; multiple captures possible per turn

---

5. Players & Turn Order

- 2–6 players per match
- All players roll a die to start; highest goes first
- Turns proceed clockwise

---

6. Dice Mechanics

- Determines how many lines a player may draw per turn
- **Edge Case:** Roll higher than remaining drawable lines = turn skipped

---

7. Game End Conditions

- Ends when no valid pizza slices remain
- **Winner:** Player with most captured slices

---

8. Entry Fees & Prize Pool

Entry Tiers: $0.25, $0.50, $1.00

Prize Distribution: - 3% → Veteran charities - 7% → Burned permanently - 3% → Daily Free Roll Pool - 10% → Weekly Top 3 players (most slices captured) - 67% → Match winner - Remainders → Weekly Jackpot

---

9. Pages

1. Home — game overview, tier selection, wallet display, free roll modal
2. Waiting Room — matchmaking by tier, player list, countdown
3. Game — active match, interactive grid, dice, captured slice effects
4. Game Over — results, payouts, share options
5. Leaderboard — weekly & lifetime stats, clickable profiles

---

10. UI Components & Assets

- **Dice:** Bouncy animation, shake + result bounce
- **PlayerCard:** PFP, FID, slices, hover/active states
- **SquareGrid:** Interactive SVG grid
- **CapturedPizzaSlice:** Spin + particle toppings
- **EntryTierSelector:** Glow on hover
- **WalletDisplay:** $PIZZA balance pulse

- **PizzaLogo:** Animated, spinning slices
- **Confetti:** Win celebration
- **FreeRollModal:** Dice selection & confetti
- **Background:** Amusement park, parallax, floating toppings
- **PrizePoolCoins:** Coin rain particle
- **Fireworks:** Spark bursts, celebratory

Assets: Lottie JSON + Rive files for all animations; SVG/PNG placeholders for static visuals

---

11. Hooks / TypeScript Interfaces

useGameState

```
interface UseGameState {
  players: PlayerID[];
  currentPlayer: PlayerID;
  rollDice: () => number;
  drawEdge: (edgeId: EdgeID) => void;
  gameOver: boolean;
  capturedSlices: Record<PlayerID, number>;
}
```

useMatchmaking

```
interface UseMatchmaking {
  joinQueue: (tier: number) => Promise<void>;
  leaveQueue: () => void;
  matchId?: string;
}
```

useWallet

```
interface UseWallet {
  address?: string;
  balance: bigint;
  approvePizza: (amount: bigint) => Promise<void>;
  enterMatch: (amount: bigint) => Promise<void>;
}
```

useFarcaster

```
interface UseFarcaster {
  viewToken: (tokenAddress: string) => void;
  composeCast: (text: string) => void;
  viewProfile: (fid: number) => void;
}
```

useLeaderboard

```
interface UseLeaderboard {
  weekly: PlayerStats[];
  lifetime: PlayerStats[];
}
```

usePlayerStats

```
interface PlayerStats {
  playerId: PlayerID;
  gamesPlayed: number;
  wins: number;
  slicesCaptured: number;
}
```

---

12. Grid Mechanics & Triangle Detection

- Nodes: `{id, x, y}`
- Edges: `{id, nodeA, nodeB, claimedBy}`
- PizzaSlices: `{id, edgeIds[3], capturedBy}`
- Algorithm: Graph-based, detect all triangles formed by new edge; capture if third side completed; grant extra turn

---

13. Onchain Contract (UUPS Upgradeable)

- **Roles:** Admin, Operator
- **Functions:**
- `enterMatch(matchId, amount)`
- `settleMatch(matchId, winner, totalPool)`
- `recordFreeRoll(fid, rollSelection, rolledNumber)`

- **Prize Pool Distribution:** 3% charities, 7% burn, 3% daily free roll, 10% weekly top 3, 67% match winner
- **Stats Recorded Onchain:** games played, wins, slices captured, lifetime $PIZZA earnings, weekly slices

---

14. Daily / Weekly Cron & Settlement

- **Daily Cron:** Settle free roll pool, pay winners, leftover → weekly jackpot, reset eligibility
- **Weekly Cron:** Rank top 3, distribute weekly jackpot, reset weekly counters
- All actions permissioned, idempotent, onchain-verifiable

---

15. Farcaster Integrations

- `viewToken($PIZZA)` — quick access
- `composeCast()` — share match win, free roll, capture streak
- `viewProfile(fid)` — tap avatars anywhere
- Free roll wins auto-prompt share
- Leaderboard ranks shareable

---

16. Animation / Motion Direction

- Unreal-level polish using Lottie/Rive + Phaser/WebGL + React Motion
- Effects:
- Pizza slice spin + particle toppings
- Dice roll bounce & shake
- Confetti, spark bursts, neon glow on win
- Parallax backgrounds with floating pizza elements
- Chain reactions for multi-slice capture

---

17. Asset Deliverables

- Pre-built grids: 4x4, 6x6, 8x8 (SVGs)
- PlayerCards (hover & active)
- Dice animations (Lottie/Rive)
- CapturedPizzaSlice animations
- FreeRollModal animations
- Confetti + particle effects
- Background images (PNG/layered SVG)
- Prize pool coins & fireworks (Lottie/Rive)
- $PIZZA token badge graphics
- Tier selector UI + hover animations

This README contains the complete blueprint for Pizza Shapes Mini App: UI, animations, Farcaster integrations, grid mechanics, TypeScript interfaces, and onchain $PIZZA contract logic.